

kArchiveManager 2.0 – Kompletní příručka pro administrátory

Obsah

- [1. Architektura a principy](#)
 - [2. Konfigurační model \(Defaults vs Overrides\)](#)
 - [3. Editace konfigurace](#)
 - [4. Cutoff a časové pásmy](#)
 - [5. Audit a protokolování](#)
 - [6. Bezpečnost a přístup \(role L1 / L2\)](#)
 - [7. SQL Agent jobs](#)
 - [8. Výkon a optimalizace](#)
 - [9. Operační funkce](#)
 - [9.1 Restore z archivu](#)
 - [9.2 Recovery uvízlých běhů](#)
 - [9.3 Zálohy archivu](#)
 - [9.4 Validace konfigurace](#)
 - [9.5 Odhady dopadu](#)
 - [9.6 Go-live kontrolní seznam](#)
 - [10. Operační záruka bezpečnosti](#)
 - [Dodatek – Klíčové tabulky a views](#)
-

1. Architektura a principy

1.1 Tři klíčové databáze

kArchiveManager pracuje se třemi databázovými vrstvami:

- **kArchiveManagerAdmin** – řídicí databáze, obsahuje konfiguraci, stav běhů, audit a monitoring. Neobsahuje archivovaná data.
- **kArchiveManagerBackups** – archivní databáze, uchovává kopie smazaných řádků oddělené do schémat podle zdrojové databáze (např. `Edge`, `KMWEBV`). Doporučen režim FULL recovery.
- **Zdrojové databáze** – provozní systémy (zákaznické), ze kterých se data archivují a mažou. Aplikace je nikdy schématicky nemodifikuje – žádné indexy, žádné DDL změny.

1.2 Tři fáze jednoho běhu (životní cyklus)

Každý proces probíhá v těchto třech explicitních fázích:

1. **SELECT (příprava)** – vybere kandidáty (řádky k archivaci) do resumovatelného keysetu podle strategie procesu (ANCHOR, TIMESTAMP, apod.).
2. **ARCHIVE (archivace)** – vloží vybrané řádky do archivní databáze (pouze pokud Mode=1 nebo Mode=2).
3. **DELETE (smazání)** – smaže zdrojové řádky JOINem na materializovaný keyset, po dávkách (pouze pokud Mode=1 nebo Mode=0).

Tímto přístupem se zajišťuje, že runner **nikdy opakovaně neskenuje velké zdrojové tabulky** s generickým `DELETE TOP` predikátem. Kandidáti se vyberou jednou, zapíší do keysetu a následné archivování/mazání probíhá přes indexované joiny.

1.3 Řetězec spuštění (oficiální v2 cesta)

Všechny produkční běhy procházejí jednotným řetězcem:

```
arch.usp_RunProfile_Prepared
├─ arch.usp_RunConfiguredProcesses_Prepared
│   └─ ANCHOR strategie:
│       └─ arch.usp_PrepareCandidates
│           └─ arch.usp_RunPreparedBatches_InWindow
│               └─ arch.usp_RunPreparedBatch
└─ TIMESTAMP strategie:
    └─ arch.usp_RunTimestampProcess
```

Pravidlo č. 1: Pro veškerý produkční provoz se používá **výhradně** `usp_RunProfile_Prepared`. V1.0 procedury neexistují v čistém zákaznickém buildu.

1.4 Tři režimy zpracování (Mode)

- **Mode 1 (archivace + mazání)** – smaže zdroj a archivuje jej; je invarianta `archived==deleted` a `Divergence=0`.
- **Mode 0 (delete-only)** – smaže zdroj bez archivace.
- **Mode 2 (copy-only)** – kopíruje do archivu bez mazání zdroje; idempotentní (re-run zkopíruje 0 nových řádků); vyžaduje PRIMARY KEY na zdrojové tabulce pro dedup.

1.5 Dvě nezávislé páky provozu

- **Run mode** – `@DryRun = 1` (náhled, nic se nemění) vs. `@DryRun = 0` (reálné zpracování).
- **AuditLevel** – `NONE` / `BATCH` / `OBJECT` / `ROW` (kolik důkazů se zapíše).

Tyto páky jsou nezávislé: dry-run obejde mazání, audit control řídí granularitu evidence.

2. Konfigurační model (Defaults vs Overrides)

2.1 Struktura – dvě úrovně

Konfigurace je organizována v dvoustupňové hierarchii:

Tabulka	Úroveň	Obsah
<code>arch.Process</code>	Šablona / Default	Výchozí nastavení procesu (společné pro všechny zdrojové databáze). Obsahuje <code>AuditLevel</code> , <code>BatchRowCount</code> , <code>CutoffMode</code> , <code>RetentionDays</code> , <code>CutoffDate</code> , <code>MaxRowsPerTransaction</code> , apod.
<code>arch.ProcessDatabase</code>	Override per-mapování	Mapování procesu × zdrojová DB × archivní DB. Každé pole zde může být <code>NULL</code> (použije se default z Process) nebo nastaveno na konkrétní hodnotu (override).
<code>arch.ObjectSpec</code>	Šablona objektu	Tabulky a sloupce, které proces zpracovává.
<code>arch.ObjectSpecDatabaseOverride</code>	Override objektu	Odchytky objektu v konkrétní zdrojové DB.

2.2 Precedence (COALESCE pravidlo)

Runtime nečte přímo tabulky výše – používá **efektivní views**, které slučují šablonu a override. Pro každou konfigurační hodnotu je aplikován vzorec:

```
COALESCE(ProcessDatabase.X, Process.X)
```

Klíčová zásada: Má-li mapování (ProcessDatabase) nastavenou hodnotu – ta se použije. Jinak se berou výchozí hodnoty z Process.

Efektivní views jsou:

- `arch.v_ProcessDatabaseEffective` – výsledná konfigurace procesu pro danou DB s informací o původu každé hodnoty (pole `*Source`).
- `arch.v_ObjectSpecDatabaseEffective` – výsledná objektová konfigurace.

2.3 Gotcha – maskování a viditelnost v konzoli

Editovat value v **Process konfiguraci** má **ŽÁDNÝ runtime efekt** pro mapování, které tuto hodnotu přepisuje. Příklad:

- Nastavíte `Process.AuditLevel = NONE`
- Ale mapování `RF_LOG2@KMWEBV` má `ProcessDatabase.AuditLevel = ROW`
- Runtime používá ROW, ne NONE

Jak to konzola řeší:

- **Sloupec "Overrides"** v Database mappings tabulce – seznam polí, která mapování přepisuje (např. "Mode, Audit, BatchRows"), nebo "-" když jsou plně zděděny.
- **Source badges** – amber ikona = field je přepsán v mapování; neutrální = zděděno z procesu.
- **Advisory v editoru procesu** – "Toto jsou výchozí hodnoty; mapování je může přepsat."

Chcete-li změnit value, kterou mapování přepisuje, **editujte mapování** (Database mappings), ne samotný proces.

3. Editace konfigurace

3.1 Configuration view – procesy, mapování, objekty, profily

Admin Console nabízí čtyři hlavní oblasti pro editaci:

a) Procesy (Processes)

Tady definujete proces jako celek – jeho název, SelectionStrategy (ANCHOR, TIMESTAMP, apod.), výchozí AuditLevel, BatchRowCount, CutoffMode, RetentionDays, CutoffDate, MaxRowsPerTransaction, apod. Pamatujte, že toto jsou **výchozí hodnoty**; mapování je mohou přepsat.

b) Database mappings

Pro každý proces zde vidíte seznam mapování (proces × zdrojová DB × archivní DB). Každé mapování má:

- Sloupec **"Overrides"** – která pole mapování přepisuje.
- Editable pole (pokud jsou přepsána) – AuditLevel, BatchRowCount, Mode, CutoffMode, RetentionDays, apod.
- Source badges (amber/neutrální) – visual hint na hierarchii.

c) Objekty (Objects)

Tabulky a sloupce, které proces zpracovává. Vidíte výběr sloupců pro archivaci/mazání.

d) Profily běhů (Run Profiles)

Deklarativní bundly – které procesy běžet a v jakém pořadí (grouping + filtering). Jedním click spustíte celý profil.

3.2 Validace a Explain plan

Každá konfigurace se validuje:

- **Configuration → Validate** – zkontroluje konzistenci všech nastavení (index requirements, foreign keys, cutoff logic, apod.). Vrací ERROR / WARN / INFO nálezy s doporučením opravy.
- **Explain plan** – vytiskne, co proces udělá: strategie, klíče, kandidáty, cutoff expression, audit level, apod. Před real run je doporučeno si explain plan projít.

3.3 Analysis & Estimates view

Zobrazuje **všechny procesy** (bez jakýchkoli whitelist filtrů). Tabulky a grafy mají 460px max-height s vnitřním scrollem, aby dlouhé seznamy nenatulovaly stránku. Odhady jsou výchozně skryty – zobrazí se kliknutím na "Show estimates".

4. Cutoff a časové pásmy

4.1 Dva režimy cutoff

Režim 1: Relativní retence (doporučeno)

- `CutoffMode = 0`
- Parametr: `RetentionDays` (kolik dní zpět si chcete ponechat).
- Cutoff se počítá jako `NOW() - RetentionDays`.
- **Výhoda:** Žádná časozonová ambiguita. Cutoff se změní každý den automaticky.
- Doporučeno pro rutinní provoz.

Režim 2: Fixní cutoff date (`CutoffMode = 1`)

- Parametr: `CutoffDate` (konkrétní datum).
- **Interpretace:** Datum je vždy `00:00:00 UTC`.
- Zdrojové timestampy jsou v místním čase (CET/CEST), normalizované přes `AT TIME ZONE`.
- **Efektivní místní hranice se posouvá se zimní/letní časem:**

Nastavený CutoffDate (UTC)	Efektivní místní hranice
2024-01-01 (zima, CET = UTC+1)	2024-01-01 01:00 místně
2024-07-01 (léto, CEST = UTC+2)	2024-07-01 02:00 místně

4.2 Live preview a konsolová podpora

Když nastavíte fixní cutoff date:

- Editor ukazuje **EFEKTIVNÍ MÍSTNÍ hranici** v reálném čase (např. "1. 1. 2024 1:00").
- Pole je označeno jako "UTC", aby bylo jasné, že zadáváte UTC čas.

- Operátor vidí skutečnou hranici bez ručních výpočtů.
- **Doporučení:** Pokud si nejste jisti, použijte relativní retenci (RetentionDays) – žádná TZ ambiguita.

5. Audit a protokolování

5.1 Čtyři úrovně auditu (AuditLevel)

Každý proces má konfigurovatelný audit, který řídí, kolik stop se zapíše o reálném běhu (DryRun je vždy bez auditu):

AuditLevel	Co se zapisuje	Náklad	Kdy používat
NONE	Jen <code>Run</code> , <code>RunItem</code> , <code>RunItemObject</code> (bez per-row evidence)	~0%	Vysokoobjemové log úklidy (RF_LOG2)
BATCH	Shrnutí per-batch (default pilot nastavení)	~0%	Rutinní monitoring
OBJECT	Per-object shrnutí (aktuálně stejné chování jako BATCH)	~0%	Budoucí rozšíření
ROW	Jeden <code>RunDocAudit</code> řádek per zpracovaný klíč/dokument	~21% výkonu	Dokladové procesy (SHIPPING, RECEIVING) nebo compliance požadavky

5.2 Auditní tabulky a views

- `arch.Run` – záznam jednoho běhu (ProcessCode, SourceDb, ArchiveDb, čas spuštění, status, counts).
- `arch.RunItem` – jedna dávka v rámci běhu (status, počet archivovaných/smazaných řádků).
- `arch.RunItemObject` – kterou tabulku/objekt jedna dávka zpracovala.
- `arch.RunDocAudit` – **pouze se zapíše pokud AuditLevel = ROW/OBJECT**. Pro každý zpracovaný klíč jeden řádek s linkem na `RunItemId`.
- `arch.v_RunDocAuditDetailed` – efektivní pohled s plným trasem (klíč, proces, zdrojová DB, archivní DB, cutoff, status, counts).

5.3 Čtyři stavy běhu (Run status)

Každý běh (`arch.Run`) i každá dávka (`arch.RunItem`) se řídí přesně **čtyřmi stavy** danými DB CHECK constraint:

Stav	Popis	Terminální?
RUNNING	Běh právě probíhá. Viditelné v Runs panelu jako aktivní.	Ne – přejde do OK / FAILED / DRYRUN
OK	Úspěšně dokončeno. <code>archived == deleted</code> , Divergence = 0.	Ano
FAILED	Chyba nebo manuálně zastaven operátorem. Data jsou konzistentní – částečně zpracované dávky se atomicky rollbackují. Příště se zpracování obnoví od místa přerušení.	Ano
DRYRUN	Náhledový běh – žádná data nebyla změněna ani smazána (pouze materializace kandidátů do WorkBatch pro ANCHOR).	Ano

Pozor: stav STOPPED neexistuje. Pokud operátor pošle pokyn k zastavení (*Stop run* v konzoli), běh skončí jako **FAILED** – ale v tabulce `arch.Run` bude vyplněno pole `CancelRequestedBy` a `CancelReason`. Podle přítomnosti tohoto pole lze odlišit „chybový“ FAILED od „manuálně zastaveného“ FAILED.

5.4 DryRun

- **DryRun = 1** – náhled bez změny dat. Zobrazí kandidáty, nic nesmaže/nearchivuje. Pro ANCHOR procesy se materializují preview klíče do `WorkBatch`; operátor si může klíče projít než spustí reálný run.
- **DryRun = 0** – reálné zpracování (s AuditLevel kontrolujícím granularitu evidence).

5.5 Change history (ConfigChangeSet)

Každá konfigurační změna v Admin Console je zaznamenána v `arch.ConfigChangeSet` :

- Kdo ji udělal (RequestedBy).
- Kdy.
- Mandatory change reason (≥6 znaků).
- Která pole se změnila.
- Status (Published – okamžitě po uložení).

Viditelný v Admin Console panelu **Change history**, exportovatelný do CSV.

6. Bezpečnost a přístup (role L1 / L2)

6.1 Rozdělení rolí: L1 a L2

Každý operátor v `arch.ConsoleOperator` má příznak `IsElevated` (bit), který určuje, ke kterým operacím má přístup:

Úroveň	IsElevated	Co může dělat	Co nemůže
L1 – první linie	0	Dashboard (přehled + zdraví), Runs (prohlížení + detail), Document lookup (kde je doklad?), monitoring stavu, čtení konfigurace, čtení auditního logu	Editovat konfiguraci, spouštět restore, přidávat legal holds, ovládat SQL Agent joby, go-live akce
L2 – druhá linie	1	Vše co L1 + editace konfigurace, validace, spuštění běhů, restore z archivu, legal holds (add/release), ovládání jobů, go-live kontrolní seznam, správa operátorů	PurgeArchive=1 (smazání archivní kopie) – výhradně DBA přes přímý SQL pod <code>karch_approver</code>

Příznak `IsElevated` se nastavuje v panelu **Configuration → Operators** (vyžaduje samo L2 přístup). V auditním záznamu (`arch.ConfigChangeSet`) je vždy zaznamenáno jméno operátora (RequestedBy).

DB role vs. konzolová úroveň

DB role	Konzolová úroveň	Klíčová oprávnění
<code>karch_viewer</code>	L1	SELECT readiness, config, runs (read-only)
<code>karch_operator</code>	L1	Stop runs, view logs
<code>karch_config_admin</code>	L2	Edit config, save operators, get auth
<code>karch_advanced_admin</code>	L2	Restore z archivu, run verify, legal holds
<code>karch_approver</code>	L2 / DBA	Nastavení retention floor, add/release legal holds, PurgeArchive (jen přímý SQL)
<code>karch_runtime</code>	Runner (ne konzole)	Execute runner procs; nemá přístup do konzole

6.2 Edit lock – tři cesty autorizace

Editace konfigurace je bráněna **edit lock** autorizací. Jsou tři koexistující cesty (ověřuje se v tomto pořadí):

Cesta 1: Windows/AD identity (produkčně doporučeno)

- Autentifikovaný Windows uživatel musí být v allowlistu `AdminConsole:AdminUsers` (např. `[ "DOMAIN\jnovak" ]`).

- Bez hesla; editor ukazuje "Editing as DOMAIN\user".
- Konfigurace:

```
"AdminConsole": {
  "WindowsAuthEnabled": true,
  "AdminUsers": ["DOMAIN\jnovak", "DOMAIN\lsmith"]
}
```

- Na IIS webu povolit **Windows Authentication**; **Anonymous Authentication** zůstane zapnuto (SPA a readiness endpointy).
- V dev (appsettings.Development.json) je WindowsAuthEnabled: false – API běží v Kestrel bez IIS Windows auth.

Cesta 2: Local operator account (bez AD)

- Pojmenování operátora přímo v arch.ConsoleOperator (databáze) – každý má username + PasswordSha256 (PBKDF2-HMAC-SHA256) + optional DisplayName + IsEnabled flag + IsElevated flag (L1/L2).
- Odemykání: username + heslo.
- RequestedBy v auditu: operátor jménem (např. "jnovak").
- **Jak generovat PasswordSha256:**

```
KArchiveManager.AdminConsole.Api.exe hash-password "the-password"
```

nebo na build serveru:

```
dotnet run --project KArchiveManager.AdminConsole.Api -- hash-password "the-password"
```

Paste výsledný PBKDF2-SHA256\$... string do konzolového editoru operátora.

- **Legacy SHA-256 hex:** Staré unsalted SHA-256 hexy se ještě ověří (back-compat), ale **regenerujte jako PBKDF2** před produkcí.

Cesta 3: Shared password (fallback / break-glass)

- AdminConsole:EditPasswordSha256 (stejný PBKDF2 formát).
- Odemykání: přijmete X-Admin-Edit-Token header; RequestedBy v auditu = "SharedPassword" (žádná per-user identity).
- Doporučeno **pouze pro dev / emergency**. V produkci raději Windows/AD nebo operator accounts.

6.3 Nové panely

a) "Poslední přihlášení" (Last login)

Read-only panel zobrazující kdo a kdy odemkl editaci. Data z arch.ConsoleLoginAudit (success + failure logins).

b) "Přístup k editaci" (Edit access)

Read-only panel ukazující:

- Windows/AD allowlist (AdminConsole:AdminUsers).
- Local operators + jejich status (Enabled /disabled) a úroveň (L1/L2).
- Shared password status (configured / not configured).

6.4 Runtime identity – least-privilege (T-33)

Governance výše řídí KDO smí měnit konfiguraci. Runtime identity řídí CO smí běžec dělat.

Dedikovaný non-sysadmin runner login je v roli `karch_runtime` s minimálními právy:

- **Admin DB:** EXECUTE na runner proc chain + VIEW DEFINITION ON SCHEMA::arch (metadata visibility).
- **Zdrojové DBs:** SELECT + DELETE na mapované tabulky (dynamic SQL nechajme); SELECT na ANCHOR candidate tabulky.
- **Archivní DB:** INSERT + SELECT + ALTER na archive schémata + CREATE TABLE, ale **NE DELETE/UPDATE** (runner nemůže archiv tamper).
- **SQL Agent job:** Job owner je non-sysadmin (ne proxy, ne sysadmin service account).
- **Run-start gate:** `arch.usp_VerifyRunnerPrivileges` ověří na startu, že runner není sysadmin/db_owner a má přesně deklarovaná práva (jinak THROW 51001).

6.5 Retention floor (T-21) – minimální retence

`arch.RetentionPolicy.MinRetentionDays` (singleton; default 0 = disabled). Operátor si nastaví minimální počet dní, které se musí uchovávat.

`arch.usp_AssertRetentionFloor` THROW 50210, pokud by běh smazal řádky mladší než floor. Kontroluje se finální cutoff, takže chytá jak příliš malý RetentionDays tak příliš nedávný CutoffDate. Ověřuje se v TIMESTAMP runneru (027), ANCHOR runneru (015) a při ANCHOR přípravě (014).

6.6 Legal hold (T-21) – blokáce smazání

- `arch.LegalHold` tabulka: (ProcessCode + optional SourceDb + HoldKey) = primární klíč procesu.
- Aktivní holdy jsou vylučovány z kandidátů na build time (fáze 014/027).
- Pokud hold přidán PO přípravě ANCHOR WorkBatch: klíč je "parkován" (status=5) a nikdy se nesmaže/nearchivuje, batch se normálně dokončí.
- Add/release: `karch_approver` role (`usp_Api_AddLegalHold` / `usp_Api_ReleaseLegalHold`), auditable CreatedBy/ReleasedBy.
- List: `usp_Frontend_GetLegalHolds`.
- Holdy fungují na Key1 group granularnosti (u composite-key procesů je over-inclusive, ale nikdy under-exclusive).

7. SQL Agent jobs

7.1 Pět klíčových jobů

PREP (příprava kandidátů)

Kandidáti jsou připraveni v `WorkBatch` + `WorkBatchKey` pro ANCHOR strategie. Bez mazání, bez archivace. Job je ve výchozím stavu **disabled** – aktivuje se z Go-live panelu nebo ručně.

RUN (hlavní běh)

Spouští `arch.usp_RunProfile_Prepared` – archivuje a maže dle konfigurace. Volá se v čase/okně. Job je ve výchozím stavu **disabled** – aktivuje se z Go-live panelu.

RECOVER (obnova zaseknutých běhů)

Viz sekci [9.2 Recovery uvízlých běhů](#). Job je doporučen spouštět každých **15 minut**, ve výchozím stavu **enabled**.

BACKUP FULL (denní záloha archivu)

Viz sekci [9.3 Zálohy archivu](#). Ve výchozím stavu **enabled**, spouští se v 01:30 UTC.

BACKUP LOG (hodinová záloha transaction logu)

Viz sekci [9.3 Zálohy archivu](#). Ve výchozím stavu **enabled**, spouští se každou hodinu.

7.2 Ownership a bezpečnost jobů

Job	Owner	Důvod
PREP, RUN	runner login (machine account)	Console potřebuje vlastnit job pro sp_start_job bez sysadmin
RECOVER, BACKUP FULL, BACKUP LOG	sa (nebo sysadmin)	Runner nemá BACKUP DATABASE ani VIEW SERVER STATE

7.3 Scheduling a run window

- `arch.RunProfile.RunWindowMinutes` – max délka běhu v minutách. Běh se sám ukončí.
- `@StopAtUtc` parametr – běh se zastaví před tímto UTC časem.
- `DelayMsBetweenBatches` – prodleva mezi dávkami (zátěž snižování).

7.4 Ovládání jobů z konzole

Panel **Configuration** → **SQL Agent Jobs** (L2 přístup) umožňuje:

- Zobrazit stav a historii jobů.
- Enable/disable job (nastaví SQL Agent enabled flag).
- Změnit scheduling (čas spuštění, interval).
- Spustit job ad-hoc (sp_start_job).

Čtení stavu jobů (GetAgentJobs, GoLiveReadiness) vyžaduje SELECT na msdb job tabulkách – zajišťuje script 059 (Console job control grants).

8. Výkon a optimalizace

8.1 Archivní primární klíč – proč jej uchovávat

Archivní tabulky zrcadlí zdrojové sloupce **včetně primárního klíče** (např. RF_LOG2.ROWID). PK není dekorace – tři features na něm závisí:

- Mode 2 idempotence:** Re-run nesmí duplikovat řádky. Dedup je po PK: druhá kopie stejných řádků vloží 0 řádků.
- Restore korektnost:** `usp_RestoreFromArchive` spoléhá na key identity aby vrátila řádky bez duplikace.
- Duplicate-archive prevention:** Unique gate (50115) a archive-as-system-of-record záruka.

8.2 Size/throughput trade-off (měřeno)

Clustered PK maintenance zvyšuje cost vkládání jak se archiv zvětšuje. Totožný 200k RF_LOG2 run, cheap-mode, **jedinou odchylkou archiv velikost na startu:**

Velikost archivní tabulky na startu	Throughput
Prázdná	~9 100 řádků/s

~5M+ nahromaděno	~2 900 řádků/s (≈3× pomalejší)
------------------	--------------------------------

To je **dominantní throughput faktor** – větší než audit level. Akceptovaný cost indexovaného, dedup-capable archivu.

8.3 Mitigace výkonu (priorita)

1. Index parking (052_archive_with_index_parking.sql) – ~1.5× rychlejší

- Deaktivuje non-clustered archivní indexy během běhu, po dokončení je znovastaví.
- Plně reversible.
- Vyžaduje `@KeepIndexesCsv` = index potřebný pro běh; THROW 50700 na prázdný list, rebuild bezpodmínečně po archive fázi.
- Používat v maintenance window pro velké loads.

2. BatchRowCount tuning

- Menší batch = menší lock footprint, ale per-batch overhead.
- Větší batch = rychlejší, ale více locks.
- S `LOCK_ESCALATION=DISABLE` na multi-index source tabulkách: ~10k–50k je dobrý range.

3. Partitioning (100M+ archiv)

- Range-partition archiv po datu.
- Inserty budou hit nejnovější partition, stará data se mohou switchnout na purge/cold storage.

8.4 NEDĚLEJTE:

- ❌ Nealterujte PK na live archive tabulkách.
- ❌ Nevytvářejte source indexy bez explicitního approval (runner je nebere, a kazí customer OLTP).

9. Operační funkce

9.1 Restore z archivu

Princip

Funkce vrací řádky z archivní DB zpět do zdrojové (produkční) DB. Jde o inverzní operaci k archivaci: místo přesunu zdroj → archiv se přesouvá archiv → zdroj.

Klíčové vlastnosti:

- **Idempotentní** – obnoví jen řádky, které ve zdroji chybí (NOT EXISTS na primárním klíči); pokud tam řádek už je, přeskočí se.
- **Transakční** – celý restore běží v jedné transakci (buď vše, nebo nic).
- **Výchozí DryRun=1** – bez explicitního `@DryRun=0` se nic nezapiše, jen se ukáže náhled.
- **Archiv zůstane nedotčen** – `@PurgeArchive` je výchozně 0; smazání archivní kopie je extra operace s dalšími vrstvami ochrany.
- **Auditovaná** – každý reálný restore (nebo purge) zapíše append-only záznam do `arch.RestoreAudit`.

Jak to technicky probíhá

```
[Archivní DB] → SELECT chybějících řádků → INSERT do [zdrojové DB]
```

Procedura (`arch.usp_RestoreFromArchive` , deploy 042) zpracovává tabulky v **obráceném DeleteOrder** (masters před children, aby FK nevyvolaly chybu), a pro tabulky s IDENTITY sloupce automaticky nastavuje `SET IDENTITY_INSERT ON/OFF` .

Parametry procedury

Parametr	Výchozí	Popis
@ProcessCode	—	Povinný – který process obnovit
@SourceDb	—	Povinný – cílová (produkční) DB
@ArchiveDb	—	Povinný – zdrojová (archivní) DB
@DryRun	1	1 = jen náhled, 0 = reálný zápis
@MaxRows	NULL	Volitelný limit řádků na tabulku
@PurgeArchive	0	1 = smaž archivní kopii po obnovení (DBA only)
@RequestedBy	—	Audit attribution (jméno operátora)

Dvouvrstvé bezpečnostní brány

Vrstva 1 – API endpoint (`POST /api/runs/restore`):

Režim	Požadovaná autorizace
DryRun = true	<code>security.IsAuthorized()</code> – standardní edit-token nebo L1+ operator
DryRun = false	<code>security.IsElevated()</code> – operátor s <code>IsElevated=1</code> (L2) nebo <code>ElevatedAdminUsers</code>

API **vždy posílá** `@PurgeArchive=false` – bez ohledu na to, co pošle klient. Smazání archivní kopie přes konzoli není možné, jde pouze přes přímý `EXEC` v SQL.

Vrstva 2 – SQL procedura (pokud by někdo zavolal přímo s `@PurgeArchive=1`):

```
IF IS_MEMBER('karch_approver') = 0 AND IS_SRVROLEMEMBER('sysadmin') = 0
    THROW 50404, 'Purging the archive requires membership in karch_approver.', 1;
IF @PurgeAuditLevel <> N'ROW'
    THROW 50405, 'Purging blocked for AuditLevel < ROW...', 1;
```

Kdo to může dělat

DB role / konzolová úroveň	DryRun náhled	Reálný restore	PurgeArchive=1
karch_viewer (L1)	Ne	Ne	Ne
karch_operator (L1)	Ne	Ne	Ne
karch_config_admin – záměrně bez EXECUTE na restore	Ne	Ne	Ne
karch_advanced_admin (L2, IsElevated=1)	Ano	Ano (přes konzoli)	Ne (API blokuje)
karch_approver / sysadmin	Ano	Ano	Ano (jen přímý SQL)

Postup pro L2 operátora

1. Přihlásit se jako L2 operátor (`IsElevated=1`).
2. V panelu **Runs** → **Restore** vybrat `ProcessCode`, `SourceDb`, `ArchiveDb`.
3. Spustit `DryRun` – ověřit v náhledu kolik řádků se vrátí a do kterých tabulek.
4. Potvrdit reálný restore (`DryRun=false`).
5. Restore zapíše záznam do `arch.RestoreAudit` (immutable).

9.2 Recovery uvízlých běhů

Princip

Procedura `arch.usp_RecoverStaleRuns` (deploy 030) automaticky detekuje a uzavírá osířelé záznamy `Run/RunItem/WorkBatch`, které neukončily svůj životní cyklus (session zrušena, server restartoval, klient se odpojil během běhu).

Co je "stale"?

- **Stale Run:** Status='RUNNING' a `StartedAt < (now - @StaleAfterMinutes)` (výchozí: 30 minut).
- **Stale WorkBatch:** Status='Running' a `LastProgressAtUtc < (now - @StaleAfterMinutes)`.

Logika obnovy

Stav RunItem	Akce na arch.Run
OK	Run.Status = 'OK', EndedAt = max(RunItem.EndedAt)
FAILED	Run.Status = 'FAILED', nastaví EndedAt a ErrorMessage
DRYRUN	Run.Status = 'DRYRUN'
RUNNING nebo NULL	Run.Status = 'FAILED' (nikdy se neinferuje úspěch z chybějících dat)

Pro WorkBatch: pokud jsou všechny klíče zpracovány → Status='Completed'; jinak → Status='Paused' (lze obnovit v příštím běhu) + reset Status=1 (claimed) klíčů na Status=0 (unclaimed).

Bezpečnostní záchrana

Recovery NIKDY neuzavírá běh, pokud je worker session stále naživu – ověřuje `sys.dm_exec_sessions`, aby se vyhnula race condition mezi legitimním dlouhým během a recovery procesem.

Parametry

Parametr	Výchozí	Popis
<code>@StaleAfterMinutes</code>	30	Práh timeout v minutách
<code>@DryRun</code>	0	1 = jen ukáže co by obnovil, 0 = aplikuje
<code>@MaxRecoveries</code>	100	Bezpečnostní limit (anti-runaway)
<code>@VerboseOutput</code>	1	Tisk diagnostiky do výstupu jobu

SQL Agent job

Job "kArchiveManager - RECOVER STALE RUNS" běží každých **15 minut**. Ve výchozím stavu **enabled**. Go-live readiness check ověřuje, že job je enabled.

9.3 Zálohy archivu

Princip

Zálohovací systém chrání `kArchiveManagerBackups` – systém-of-record pro všechny přesunuté řádky. Doporučen FULL recovery model na archivní DB.

Strategie FULL + LOG

Typ	Frekvence	Výchozí čas	Retence
FULL backup	Denně	01:30 UTC	35 dní
LOG backup	Každou hodinu	každou celou hodinu	8 dní

Co každý typ dělá

FULL backup:

- Vytvoří .bak soubor: {BackupRoot}\kArchiveManagerBackups_FULL_yyyyMMdd_HHmss.bak
- WITH COMPRESSION, CHECKSUM, INIT, STATS = 10
- Ihned po zápisu: RESTORE VERIFYONLY s CHECKSUM (ověří integritu).
- Vyčistí staré FULL backupy starší než 35 dní.

LOG backup:

- Kontrola: pouze pokud DB je v FULL recovery modelu.
- Vytvoří .trn soubor: {BackupRoot}\kArchiveManagerBackups_LOG_yyyyMMdd_HHmss.trn
- WITH COMPRESSION, CHECKSUM
- Vyčistí staré LOG backupy starší než 8 dní.

Konfigurace (parametry scriptu 048)

Parametr	Výchozí	Popis
@BackupRoot	CHANGE-ME	Fyzická cesta pro zálohy (Agent service account musí mít write přístup)
@RetentionDaysFull	35	Dny uchování FULL backupů
@RetentionDaysLog	8	Dny uchování LOG backupů
@FullDailyTime	013000	Čas FULL backupu (HHMMSS formát)
@LogEveryMinutes	60	Interval LOG backupu v minutách

Poznámka: Toto je fallback zálohovací řešení. Pokud zákazník používá Ola Hallengren nebo jiný maintenance plán, integrujte tam – nepoužívejte oba systémy souběžně.

Go-live ověření zálohy

Go-live readiness check ověřuje, že FULL backup proběhl v posledních 2 dnech. Pokud ne, panel zobrazí WARN.

9.4 Validace konfigurace

Princip

Procedura arch.usp_ValidateConfiguration provede 50+ kontrol konfigurace a vrátí nálezy s kategorizovanou závažností. Spouští se z panelu **Configuration** → **Validate** nebo automaticky při Go-live readiness.

Výstupní formát

Sloupec	Popis
Severity	'ERROR' / 'WARN' / 'INFO'
ProcessCode, SourceDb, ArchiveDb	Kde byl nález

Finding	Popis problému
SuggestedSql	Konkrétní SQL oprava (kde je deterministická)

Hlavní oblasti kontrol

Oblast	Příklady	Severity
Povinné objekty	Proces bez ObjectSpec; mapování bez aktivní zdrojové DB	ERROR
Redundantní override	ProcessDatabase kopíruje přesně Process default	WARN
ANCHOR strategie	Anchor bez schema/table/expr	ERROR
TIMESTAMP strategie	Bez TimestampExpr; DeleteMode nekompatibilní	ERROR
Časové pásmo	AT TIME ZONE s neznámou zónou (není v sys.time_zone_info)	WARN
Cheap-mode	CandidateSelectExpr bez CandidateWhereSql (mode se neaktivuje)	WARN
Databáze	SourceDb/ArchiveDb neexistuje; Source==Archive pro Mode=1/2	ERROR
Tabulky	Source/Archive tabulka neexistuje; mismatch ObjectSpec.DeleteMode	ERROR
Delete ochrana	AllowDeleteWithoutArchive=0 + Mode=0 kombinace	ERROR

Návratový kód

- 0 – OK, žádné ERROR nálezy
- 1 – existují ERROR nálezy; RUN CONFIGURED job odmítne spustit dokud nejsou opraveny

Parametry

- @ProcessCode (NULL = všechny procesy)
- @SourceDb (NULL = všechny zdrojové DBs)

9.5 Odhady dopadu

Princip

Procedura arch.usp_Api_EstimateNextRunImpact (deploy 060) odhaduje kolik dat by příští běh přesunul (řádky + MB), projekční nárůst archivu, tlak na log a plánovací kalkulaci s safety faktorem – per-process i per-tabulku.

Odhady jsou výchozně skryty

V panelu Analysis & Estimates jsou odhady výchozně skryté (mohou trvat sekundy pro velké databáze). Zobrazí se kliknutím na "Show estimates".

Parametry

Parametr	Výchozí	Popis
@SourceDb	NULL	NULL = všechny enabled zdrojové DBs
@ArchiveDb	NULL	NULL = všechny
@ProcessCode	NULL	NULL = všechny enabled procesy
@ArchiveGrowthFactor	1.20	20% režie na PK/indexy v archivu
@LogMultiplier	3.00	3× payload pro log pressure
@SafetyFactor	1.30	30% bezpečnostní buffer

Výstupní metriky (per-process)

Metrika	Co znamená
SourceRowsTotalAcrossConfiguredTables	Celkové řádky v mapovaných tabulkách
OneToOnePayloadMB	Odhadovaný objem dat k přesunu
RoughArchiveGrowthMB	Nárůst archivní DB (jen Mode=1)
RoughLogPressureMB	Odhadovaný tlak na transaction log
PlanningMB_WithSafety	Plánovaný MB s bezpečnostním bufferem

9.6 Go-live kontrolní seznam

Princip

Procedura `arch.usp_Frontend_GoLiveReadiness` (deploy 049) serializuje produkční audit jako go-live gate. Zobrazí se v panelu **Go-live** v konzoli (L2 přístup).

Vrací jednu řadu na check s Category, CheckName, Severity (OK/INFO/WARN/FAIL), Detail a Recommendation.

Pět kategorií kontrol

Kategorie	Kontroly
A. Privileges	Žádní over-privileged principálové Destruktivní procs grantovány správným rolím (restore → karch_advanced_admin)
B. Cleanliness	Žádné legacy_v1 / v1-stub procedury Žádné relic tabulky (RowCountSnapshot, *_Backup_*)
C. Audit Governance	Per-document audit coverage (Mode=1 doklady by měly mít AuditLevel=ROW) Audit trail tamper-resistant (DENY UPDATE/DELETE na arch.RunDocAudit)
D. Config/Timezone Gate	Všechny enabled Mode=1 cutoff expression mají AT TIME ZONE (jinak REAL runs zablokuje THROW 50200)
E. Operations	Žádné ERROR řádky v arch.v_OperationalHealth Failure alerting nakonfigurován (Database Mail + operator + job notify) Archivní DB má FULL backup v posledních 2 dnech RECOVER STALE RUNS job je enabled

Výsledný verdikt

- **FAIL** – go-live blokátor; je nutné opravit před spuštěním produkčních jobů.
- **WARN** – upozornění; lze spustit ale doporučeno řešit.
- **OK** – vše v pořádku.

10. Operační záruka bezpečnosti

Tyto záruky byly ověřeny live testováním (~5M řádků, všechny varianty).

10.1 Mode 1 invarianta (archivace + mazání)

- `archived == deleted` v každé dávce.
- `Divergence = 0` (GlobalDivergence, DivergentMode1, BadRuns všechny = 0).
- **Mechanismus:** `DELETE ... OUTPUT INTO archive`, atomicky per dávce. Zdrojový řádek je archivován a smazán v jedné transakci.

10.2 Lock-bezpečnost na zdrojovém serveru

- **Candidate selection (PREP):** Používá `WITH (NOLOCK)`. Drží jen **OBJECT Sch-S** na zdroji (schema-stability lock), **NULA S/IS/X/U data locks**. Sch-S nikdy neblokuje INSERT/UPDATE/DELETE → candidate selection nemůže blokovat OLTP aplikaci.
- **DELETE (RUN):** X-locks (mutace, v okně); malé, bounded batches. `LOCK_ESCALATION=DISABLE` na multi-index tabulkách zabraňuje escalation.
- **Proof (lock probe):** Během ~40s NOLOCK candidate scan Edge.RF_LOG2, session drží jen Sch-S – nula data locks.

10.3 Concurrency (dva běhy, totožné mapování)

- AppLock serializuje dva běhy stejného mapování (TIMESTAMP a ANCHOR).
- Jeden běží, druhý je čistě odmítnut (nenastane double-archiving, deadlock).

10.4 Retention floor (50210)

- Blokuje smazání dat mladších než policy minimum.
- DryRun je vyjmut (preview není omezena).

10.5 Legal hold

- Držená data jsou vyloučena/parkována.
- Nikdy se nearchivují/nemažou, dokud nejsou uvolněna.

10.6 Crash-consistency (kilování běhu mid-batch)

- In-flight batch se atomicky rollbackuje (nic se nestratí, nic se neduplikuje).
- `arch.usp_RecoverStaleRuns` zavře osiřelé běhy.
- Zbytek je resumovatelný.

10.7 Mode tabulka

Mode	Archivace	Mazání	Idempotence	PK na zdroji
1	✓ Archivuje	✓ Maže	Ne (archiv=source)	Doporučeno
0	✗ Ne	✓ Maže	Ne	Doporučeno
2	✓ Kopíruje	✗ Ne (nikdy)	✓ Ano (dedup po PK)	✓ Vyžaduje

10.8 Shrnutí – co si vzít do paměti

- ✓ Mode 1 je atomic a safe (archived==deleted, Div=0).
- ✓ Candidate selection nikdy neblokuje zdrojovou aplikaci (NOLOCK, Sch-S only).
- ✓ Dva běhy stejného mapování se serializují (žádný race condition).
- ✓ Crash mid-batch je bezpečný (atomic rollback, resumovatelný zbytek).
- ✓ Retention floor a legal hold zabraňují případným chybám operátora.
- ✓ Archive PK je nezbytný (dedup, restore, duplicate-prevention).
- ⚠ Archive size (ne audit) je dominant throughput faktor; index parking mitiguje.

Dodatek – Klíčové tabulky a views

Konfigurace

- `arch.Process` – definice procesu (defaults).
- `arch.ProcessDatabase` – per-mapping override.
- `arch.ObjectSpec` – objekty (tabulky) v procesu.
- `arch.ObjectSpecDatabaseOverride` – per-DB odchylky objektu.
- `arch.RunProfile` – deklarativní bundly procesů.

Runtime & Audit

- `arch.Run` – záznam běhu (Status: RUNNING/OK/FAILED/DRYRUN).
- `arch.RunItem` – dávka v běhu.
- `arch.RunItemObject` – objekt zpracovaný dávkou.
- `arch.RunDocAudit` – per-row evidence (pouze AuditLevel=ROW/OBJECT).
- `arch.v_RunDocAuditDetailed` – plný tras.
- `arch.RestoreAudit` – immutable záznam každého restore operace.

Governance & Bezpečnost

- `arch.ConfigChangeSet` – history editací (who, when, reason, fields).
- `arch.ConsoleLoginAudit` – login log (success/failure).
- `arch.ConsoleOperator` – operátoři konzole (IsElevated = L1/L2).
- `arch.RetentionPolicy` – retention floor (MinRetentionDays).
- `arch.LegalHold` – blokace smazání.

Efektivní views (config resolution)

- `arch.v_ProcessDatabaseEffective` – výsledná konfigurace (Process defaults + ProcessDatabase overrides).
- `arch.v_ObjectSpecDatabaseEffective` – výsledná objektová konfigurace.

Chybové kódy (rychlá referenční karta)

Kód	Název	Popis	Kdo řeší
50200	TZ cutoff chyba	Cutoff není převeden na UTC (chybí AT TIME ZONE)	L2 – opravit CandidateSelectExpr / TimestampExpr
50210	Retention floor	Cutoff je příliš nedávný – pod bezpečné minimum	L2 – zvýšit RetentionDays nebo CutoffDate
50404	PurgeArchive role	Volán PurgeArchive bez karch_approver role	DBA
50405	PurgeArchive AuditLevel	PurgeArchive vyžaduje AuditLevel=ROW	DBA
51000	Config neplatná	Konfigurace pro běh není platná nebo aktivní	L2 – Configuration + Validation
51001	Runner privilege gate	Runner nemá oprávnění (časté po obnově zdrojové DB)	L2 – znovu spustit 053 + 054
50004	Legacy proc	Volána blokována legacy procedura	info / L2

Poslední aktualizace: 2026-06-26 | **Verze:** kArchiveManager 2.0 Admin Console | **Jazyk:** Czech